Jour 3 : standalone, internationalisation, formulaires, tests, rendu serveur

Copier un code : cliquer dans la case blanche à droite du titre du bloc, puis **Cmd+A** et **Cmd+C** (sous Windows : **Ctrl+A** et **Ctrl+C**). Le code copié est exact, avec son indentation.

Plan de la journée

Horaire	Partie
9 h 00 – 9 h 15	Standalone : une application sans modules
9 h 15 – 9 h 55	Traduire l'application
9 h 55 – 10 h 30	Un formulaire qui refuse une saisie incomplète
10 h 30 – 10 h 45, 11 h 00 – 11 h 15	Un formulaire construit à partir d'une description
10 h 45 – 11 h 00	pause
11 h 15 – 11 h 45	Tester un pipe, un service, un guard, un composant
11 h 45 – 12 h 15	Corriger un défaut en écrivant le test d'abord (TDD)
12 h 15 – 12 h 30	Mesurer la couverture des tests
12 h 30 – 13 h 30	déjeuner
13 h 30 – 13 h 45	Tester un parcours complet avec Playwright
13 h 45 – 14 h 30	Envoyer des pages déjà rendues par le serveur
14 h 30 – 14 h 45	Séquence 17 : bilan
14 h 45 – 15 h 15	Rattrapage
15 h 15 – 15 h 30	pause
15 h 30 – 16 h 45	TP Cuisto, jour 3
Si le temps le permet	Pour aller plus loin

9 h 00 – 9 h 15

Standalone : une application sans modules

Module 4, « Angular standalone ». Point de départ, pour qui en a besoin : `shopflow-debut-jour-3.zip` et `cuisto-debut-jour-3.zip` (dossier `ressources/points-de-reprise/`), mêmes commandes que la reprise du jour 2.

1. La notion

Objectif : savoir ce qu'est un composant standalone, et regrouper toute la configuration de ShopFlow dans une seule fonction.

Pourquoi : avec les NgModules, chaque composant devait être déclaré dans un module, et chaque module importer les autres : beaucoup de code sans valeur métier.

Dans la réalité : en reprenant une application Angular 14, on trouve des dizaines de modules. Depuis Angular 19, tout est standalone par défaut, et la migration se fait avec une commande d'Angular (`ng g @angular/core:standalone`).

Le même composant, avec et sans module

```
// Avec NgModule (code existant)
@Component({ selector: 'app-promo-banner', standalone: false, template: `…{{ montant | prix }}…` })
export class PromoBanner {}

@NgModule({
  declarations: [PromoBanner, PrixPipe],
  imports: [CommonModule],
  exports: [PromoBanner],
})
export class LegacyModule {}

// Standalone (Angular 19 et suivants)
@Component({ selector: 'app-promo-banner', imports: [PrixPipe], template: `…{{ montant | prix }}…` })
export class PromoBanner {}
```

2. La démonstration

Dans `src/app/app.config.ts`, la configuration de ShopFlow est une liste de fournisseurs (routeur, client HTTP, intercepteur, langue) qu'une seconde application devrait recopier.

3. La correction

libs/shared/data-access/src/lib/provide-shopflow.ts

```
import { EnvironmentProviders,
```

```ts
import { EnvironmentProviders, makeEnvironmentProviders } from '@angular/core';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { authInterceptor } from './services/auth-interceptor';

/**
 * Tout ce dont les services de ShopFlow ont besoin pour fonctionner : client HTTP et ajout du jeton.
 *
 * Une application qui utilise `@shopflow/data-access` appelle `provideShopflow()` dans sa configuration.
 */
export function provideShopflow(): EnvironmentProviders {
  return makeEnvironmentProviders([
    provideHttpClient(withInterceptors([authInterceptor])),
  ]);
}
```

libs/shared/data-access/src/index.ts

```
export * from
```

```ts
export * from './lib/services/product-service';
export * from './lib/services/cart-service';
export * from './lib/services/auth-service';
export * from './lib/services/order-service';
export * from './lib/services/auth-interceptor';
export * from './lib/provide-shopflow';
```

src/app/app.config.ts

```
import { ApplicationConfig, LOCALE_ID,
```

```ts
import { ApplicationConfig, LOCALE_ID, provideBrowserGlobalErrorListeners } from '@angular/core';
import { registerLocaleData } from '@angular/common';
import localeFr from '@angular/common/locales/fr';
import { provideRouter, withComponentInputBinding } from '@angular/router';
import { provideShopflow } from '@shopflow/data-access';
import { routes } from './app.routes';

registerLocaleData(localeFr);

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes, withComponentInputBinding()),
    provideShopflow(),
    { provide: LOCALE_ID, useValue: 'fr' },
  ],
};
```

4. Le résultat

Aucun NgModule dans le projet ; `app.config.ts` appelle `provideShopflow()` ; une autre application du dépôt obtiendrait le même socle avec la même ligne.

9 h 15 – 9 h 55

Traduire l’application

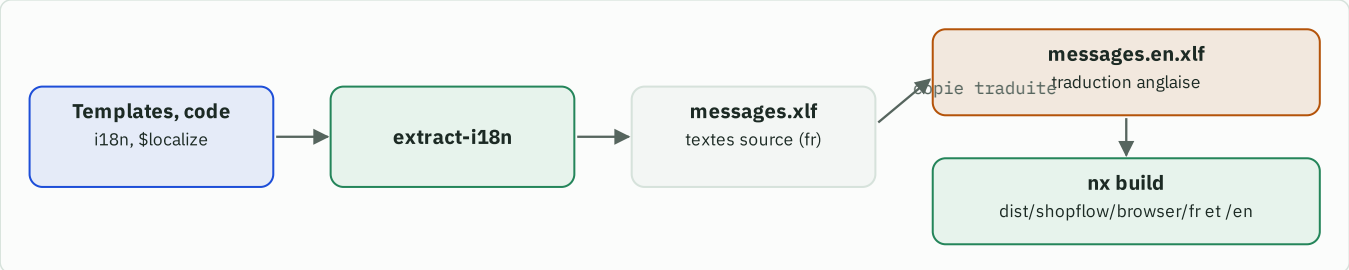
Module 5, « Internationalisation ».

1. La notion

Objectif : produire une version anglaise de ShopFlow, avec les prix et les dates au format de chaque langue.

Pourquoi : tous les textes sont écrits en dur dans les templates : impossible d’ouvrir la boutique à des clients anglophones.

Dans la réalité : une boutique de Tourcoing vend aussi en Belgique, aux Pays-Bas, au Royaume-Uni. Chaque langue est une version construite à part, servie à sa propre adresse (`/fr/` , `/en/`).



Le circuit des textes : marquer, extraire, traduire, construire une version par langue.

2. La démonstration

Le catalogue affiche « 194 produits » en dur dans `src/app/pages/catalogue/catalogue.html` : aucune version anglaise possible.

3. La correction

Installer et configurer les langues :

Terminal : shopflow — npx nx add @angular/localize

```
npx nx add @angular/localize
npx nx g @angular/localize:ng-add
```

project.json (extrait) — {

```
{
  "name": "shopflow",
  "i18n": {
    "sourceLocale": "fr",
    "locales": {
      "en": { "translation": "src/locale/messages.en.xlf" }
    }
  },
  "targets": {
    "build": {
      "configurations": {
        "production": { "localize": true },
```

```
        "en": { "localize": ["en"] }
      }
    },
    "serve": {
      "configurations": {
        "en": { "buildTarget": "shopflow:build:development,en" }
      }
    },
    "extract-i18n": {
      "executor": "@angular/build:extract-i18n",
      "options": { "buildTarget": "shopflow:build" }
    }
  }
}
```

Marquer les textes :

src/app/layout/header/header.html

```
<header class="entete">
  <a class="logo" routerLink="/">ShopFlow</a>
  <nav class="nav">
    <a routerLink="/catalogue" routerLinkActive="actif" i18n="@@nav.catalogue">Catalogue</a>
    <a routerLink="/panier" routerLinkActive="actif" i18n="@@nav.panier">Panier ({{ nbArticles() }})</a>
    <a routerLink="/mes-commandes" routerLinkActive="actif" i18n="@@nav.commandes">Mes commandes</a>
    <a routerLink="/admin" routerLinkActive="actif" i18n="@@nav.admin">Administration</a>
  </nav>
  <div class="entete-actions">
    @if (utilisateur(); as u) {
      <span class="info" i18n="@@entete.bonjour">Bonjour {{ u.firstName }}</span>
      <button class="bouton bouton-secondaire" type="button" (click)="deconnecter()"
i18n="@@entete.deconnexion">Déconnexion</button>
    } @else {
      <a class="bouton" routerLink="/connexion" i18n="@@entete.connexion">Connexion</a>
    }
  </div>
</header>
```

src/app/pages/catalogue/catalogue.html

```
<h1 i18n="Titre de la page du catalogue@@catalogue.titre">Catalogue</h1>
@if (page(); as page) {
  <p class="sous-titre" i18n="@@catalogue.total">{page.total, plural, =0 {Aucun produit} =1 {1 produit} other
{{{ page.total }} produits}}</p>
  <div class="barre-outils">
    <input class="champ recherche" type="search" placeholder="Rechercher un produit…" i18n-
placeholder="@@catalogue.recherche" (input)="onRecherche($event)" />
    <select class="champ" (change)="onCategorie($event)">
      <option value="" i18n="@@catalogue.toutes">Toutes les catégories</option>
      @for (c of categories(); track c.slug) {
        <option [value]="c.slug">{{ c.name }}</option>
      }
    </select>
    <select class="champ" (change)="onTri($event)">
```

```
      <option value="nom" i18n="@@catalogue.tri.nom">Trier par nom</option>
      <option value="prix-croissant" i18n="@@catalogue.tri.croissant">Prix croissant</option>
      <option value="prix-decroissant" i18n="@@catalogue.tri.decroissant">Prix décroissant</option>
    </select>
  </div>
  <div class="grille">
    @for (p of produits(); track p.id) {
      <sf-product-card [product]="p" [prioritaire]="$index < 4" (ajout)="panier.add($event)" />
    } @empty {
      <p class="info" i18n="@@catalogue.vide">Aucun produit ne correspond à votre recherche.</p>
    }
  </div>
} @else {
  <p class="info" i18n="@@catalogue.chargement">Chargement du catalogue…</p>
}
```

Extraire les textes et les traduire :

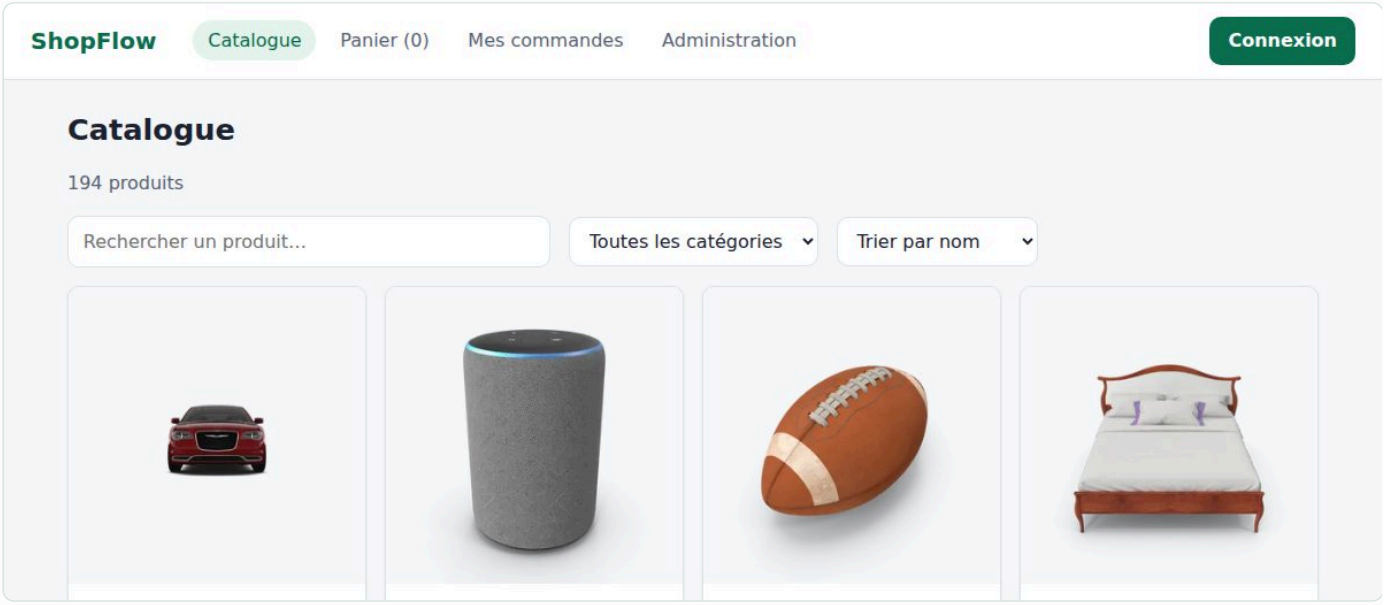
Terminal : shopflow

```
npx nx run shopflow:extract-i18n --output-path=src/locale
cp src/locale/messages.xlf src/locale/messages.en.xlf
```

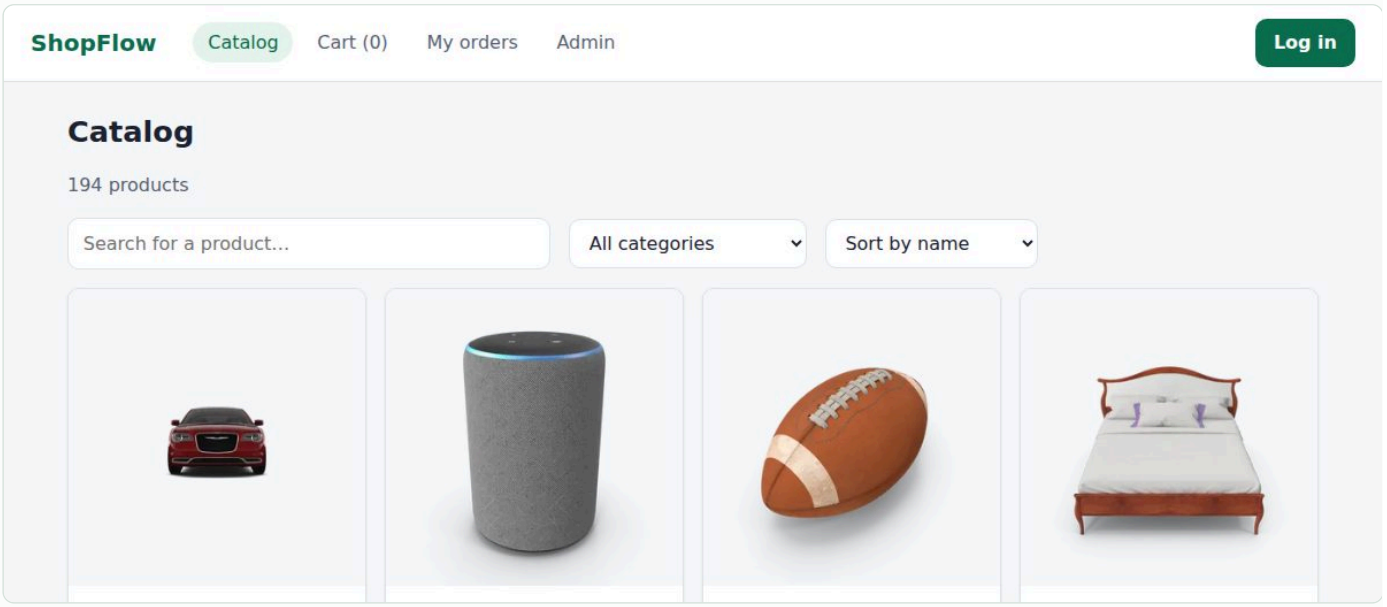
src/locale/messages.en.xlf (extrait) (extrait)

```
<?xml version="1.0" encoding="UTF-8" ?>
<xliff version="1.2" xmlns="urn:oasis:names:tc:xliff:document:1.2">
  <file source-language="fr" target-language="en" datatype="plaintext" original="ng2.template">
    <body>
      <trans-unit id="carte.ajouter" datatype="html">
        <source>Ajouter au panier</source>
        <target>Add to cart</target>
        <context-group purpose="location">
          <context context-type="sourcefile">libs/shared/ui/src/lib/product-card/product-card.html</context>
          <context context-type="linenumber">16,18</context>
        </context-group>
      </trans-unit>
      <trans-unit id="nav.panier" datatype="html">
        <source>Panier (<x id="INTERPOLATION" equiv-text="{{ nbArticles() }}"/>)</source>
        <target>Cart (<x id="INTERPOLATION" equiv-text="{{ nbArticles() }}"/>)</target>
      </trans-unit>
      <trans-unit id="catalogue.total" datatype="html">
        <source>{VAR_PLURAL, plural, =0 {Aucun produit} =1 {1 produit} other {<x id="INTERPOLATION"/>
produits}}</source>
        <target>{VAR_PLURAL, plural, =0 {No products} =1 {1 product} other {<x id="INTERPOLATION"/> products}}
</target>
      </trans-unit>
      <!-- … -->
```

4. Le résultat



Version française : « 194 produits », 27 845,79 €.



Version anglaise : « 194 products », €27,845.79 : la monnaie suit la langue.

9 h 55 – 10 h 30

Un formulaire qui refuse une saisie incomplète

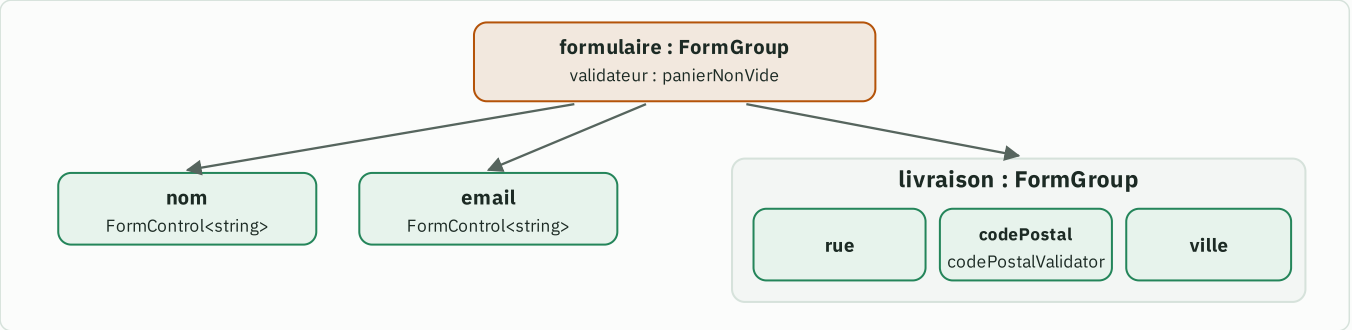
Module 6, « Formulaires réactifs, validation, erreurs personnalisées ».

1. La notion

Objectif : que le formulaire de commande vérifie chaque champ et affiche une erreur claire.

Pourquoi : le formulaire de départ accepte un envoi vide : la commande part sans adresse.

Dans la réalité : une commande sans adresse ou avec un code postal faux, c'est un colis perdu et un appel au service client. Le contrôle se fait dans le navigateur pour le confort, et de nouveau sur le serveur pour la sécurité.



Le modèle du formulaire de commande : un groupe, deux champs, un sous-groupe pour l'adresse.

2. La démonstration

Sur `/commande`, cliquer sur « Commander » sans rien remplir : la commande est envoyée.

3. La correction

src/app/pages/checkout/checkout.ts — `import { Component, inject } from`

```ts
import { Component, inject } from '@angular/core';
import { CurrencyPipe } from '@angular/common';
import { NonNullableFormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';
import { Router } from '@angular/router';
import { AuthService, CartService, OrderService } from '@shopflow/data-access';
import { codePostalValidator, panierNonVide } from './validators';

@Component({
  selector: 'app-checkout',
  imports: [CurrencyPipe, ReactiveFormsModule],
  templateUrl: './checkout.html',
})
export class Checkout {
  private readonly cart = inject(CartService);
  private readonly orderService = inject(OrderService);
  private readonly auth = inject(AuthService);
  private readonly router = inject(Router);
```

```
  private readonly fb = inject(NonNullableFormBuilder);

  protected readonly total = this.cart.total;

  protected readonly formulaire = this.fb.group(
    {
      nom: ['', [Validators.required, Validators.minLength(2)]],
      email: ['', [Validators.required, Validators.email]],
      livraison: this.fb.group({
        rue: ['', Validators.required],
        codePostal: ['', [Validators.required, codePostalValidator]],
        ville: ['', Validators.required],
      }),
    },
    { validators: panierNonVide(this.cart) },
  );

  protected valider(): void {
    if (this.formulaire.invalid) {
      this.formulaire.markAllAsTouched();
      return;
    }
    const { nom, livraison } = this.formulaire.getRawValue();
    console.info(`Commande de ${nom}, livraison à ${livraison.ville}`);
    this.orderService.create(this.auth.userId ?? 1, this.cart.lignes()).subscribe((commande) => {
      this.cart.clear();
      this.router.navigate(['/commande/confirmation'], { queryParams: { numero: commande.id, total:
commande.total } });
    });
  }
}
```

src/app/pages/checkout/checkout.html <h1>Commande</h1>

```
<h1>Commande</h1>
<form class="formulaire" [formGroup]="formulaire" (ngSubmit)="valider()">
  @if (formulaire.hasError('panierVide')) {
    <p class="alerte">Votre panier est vide : ajoutez au moins un produit.</p>
  }
  @let nom = formulaire.controls.nom;
  <div class="groupe">
    <label for="nom">Nom complet</label>
    <input id="nom" class="champ" formControlName="nom" [class.invalide]="nom.touched && nom.invalid" />
    @if (nom.touched && nom.hasError('required')) {
      <p class="erreur">Le nom est obligatoire.</p>
    } @else if (nom.touched && nom.hasError('minlength')) {
      <p class="erreur">Au moins {{ nom.getError('minlength').requiredLength }} caractères.</p>
    }
  </div>
  @let email = formulaire.controls.email;
  <div class="groupe">
    <label for="email">E-mail</label>
    <input id="email" class="champ" type="email" formControlName="email" [class.invalide]="email.touched &&
```

```
email.invalid" />
    @if (email.touched && email.hasError('required')) {
      <p class="erreur">L'e-mail est obligatoire.</p>
    } @else if (email.touched && email.hasError('email')) {
      <p class="erreur">Adresse e-mail invalide.</p>
    }
  </div>
  <fieldset class="groupe" formGroupName="livraison">
    <legend>Adresse de livraison</legend>
    @let livraison = formulaire.controls.livraison.controls;
    <label for="rue">Rue</label>
    <input id="rue" class="champ" formControlName="rue" [class.invalide]="livraison.rue.touched &&
livraison.rue.invalid" />
    @if (livraison.rue.touched && livraison.rue.hasError('required')) {
      <p class="erreur">La rue est obligatoire.</p>
    }
    <label for="codePostal">Code postal</label>
    <input id="codePostal" class="champ" inputmode="numeric" formControlName="codePostal"
[class.invalide]="livraison.codePostal.touched && livraison.codePostal.invalid" />
    @if (livraison.codePostal.touched && livraison.codePostal.hasError('required')) {
      <p class="erreur">Le code postal est obligatoire.</p>
    } @else if (livraison.codePostal.touched && livraison.codePostal.hasError('codePostal')) {
      <p class="erreur">Cinq chiffres, par exemple 59200.</p>
    }
    <label for="ville">Ville</label>
    <input id="ville" class="champ" formControlName="ville" [class.invalide]="livraison.ville.touched &&
livraison.ville.invalid" />
    @if (livraison.ville.touched && livraison.ville.hasError('required')) {
      <p class="erreur">La ville est obligatoire.</p>
    }
  </fieldset>
  <p class="total">Montant à payer : {{ total() | currency: 'EUR' }}</p>
  <button class="bouton" type="submit" [disabled]="formulaire.hasError('panierVide')">Valider la
commande</button>
</form>
```

Une règle propre au métier (code postal français, panier non vide) :

src/app/pages/checkout/validators.ts

```
import { AbstractControl,
```

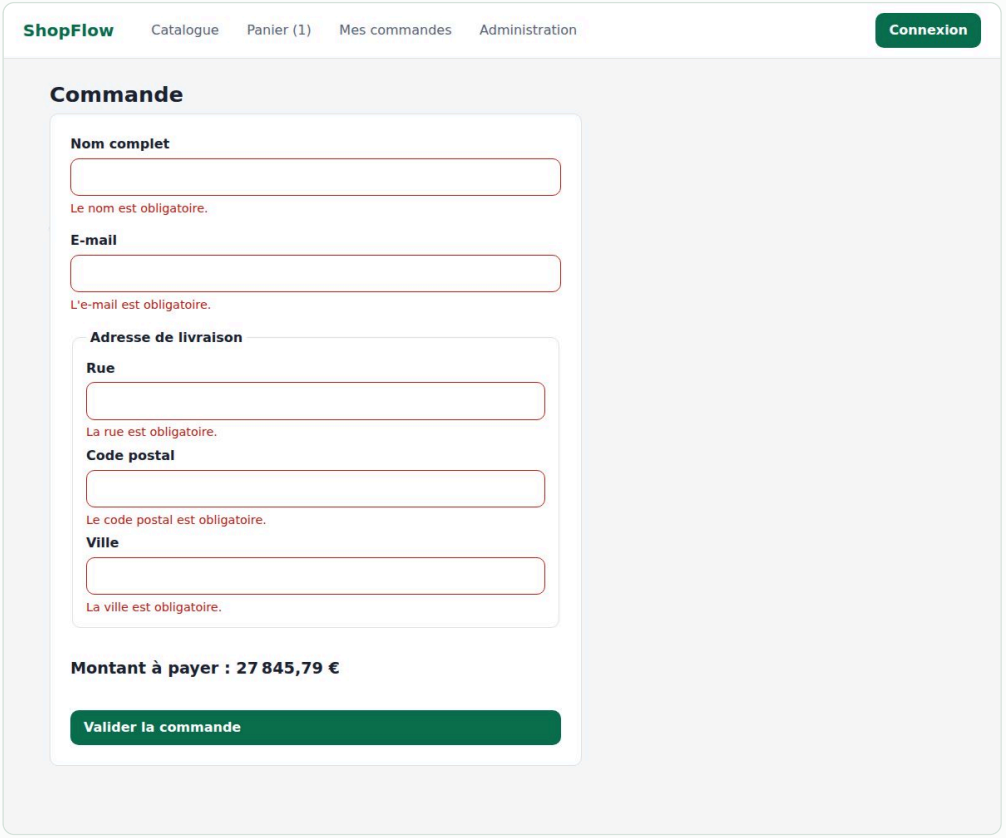
```
import { AbstractControl, ValidationErrors, ValidatorFn } from '@angular/forms';
import { CartService } from '@shopflow/data-access';

/** Code postal français : cinq chiffres. Un champ vide est laissé à Validators.required. */
export function codePostalValidator(controle: AbstractControl<string>): ValidationErrors | null {
  const valeur = controle.value;
  if (!valeur) {
    return null;
  }
  return /^\d{5}$/.test(valeur) ? null : { codePostal: { valeur } };
}

/** Erreur « panierVide » sur le formulaire entier si le panier ne contient rien. */
export function panierNonVide(panier: CartService): ValidatorFn {
```

```
  return () => (panier.lignes().length > 0 ? null : { panierVide: true });
}
```

4. Le résultat



Envoi d'un formulaire vide : chaque champ affiche son erreur.

10 h 30 – 10 h 45, 11 h 00 – 11 h 15

Un formulaire construit à partir d’une description

Module 6, « Création dynamique de template ».

1. La notion

Objectif : générer le formulaire produit de l’administration à partir d’une liste de champs, au lieu de l’écrire à la main.

Pourquoi : chaque nouveau champ oblige à modifier le template, le composant et la validation.

Dans la réalité : dans un back-office, les fiches changent souvent (un nouveau champ « garantie », une étiquette « nouveauté ») : une description des champs suffit à faire évoluer le formulaire.

2. La démonstration

Le formulaire produit n’existe pas encore : il faudrait écrire chaque champ dans le template.

3. La correction

src/app/admin/champs-produit.ts

```ts
/** Description d'un champ du formulaire produit. */
export interface ChampProduit {
  cle: 'title' | 'description' | 'price' | 'category' | 'stock';
  libelle: string;
  type: 'texte' | 'zone' | 'nombre' | 'liste' | 'quantite';
  requis?: boolean;
  min?: number;
  options?: string[];
}

/** Le formulaire produit, décrit comme des données : on ajoute un champ en ajoutant une ligne. */
export const CHAMPS_PRODUIT: ChampProduit[] = [
  { cle: 'title', libelle: 'Nom du produit', type: 'texte', requis: true },
  { cle: 'description', libelle: 'Description', type: 'zone' },
  { cle: 'price', libelle: 'Prix (€)', type: 'nombre', requis: true, min: 0.01 },
  { cle: 'category', libelle: 'Catégorie', type: 'liste', requis: true, options: ['beauty', 'fragrances',
'furniture', 'groceries', 'smartphones'] },
  { cle: 'stock', libelle: 'Stock initial', type: 'quantite', min: 0 },
];
```

src/app/admin/product-form.ts

```ts
import { Component, inject, signal } from '@angular/core';
import { FormArray, FormControl, FormGroup, ReactiveFormsModule, ValidatorFn, Validators } from
'@angular/forms';
import { ProductService } from '@shopflow/data-access';
import { Quantite } from '@shopflow/ui';
import { CHAMPS_PRODUIT, ChampProduit } from './champs-produit';
```

```
function validateurs(champ: ChampProduit): ValidatorFn[] {
  const liste: ValidatorFn[] = [];
  if (champ.requis) liste.push(Validators.required);
  if (champ.min !== undefined) liste.push(Validators.min(champ.min));
  return liste;
}

function construire(champs: ChampProduit[]): FormGroup<Record<string, FormControl<string | number>>> {
  const controles: Record<string, FormControl<string | number>> = {};
  for (const champ of champs) {
    const initiale = champ.type === 'nombre' || champ.type === 'quantite' ? 0 : '';
    controles[champ.cle] = new FormControl<string | number>(initiale, { nonNullable: true, validators:
validateurs(champ) });
  }
  return new FormGroup(controles);
}

@Component({
  selector: 'app-product-form',
  imports: [Quantite, ReactiveFormsModule],
  templateUrl: './product-form.html',
})
export class ProductForm {
  private readonly productService = inject(ProductService);
  protected readonly champs = CHAMPS_PRODUIT;
  protected readonly formulaire = construire(this.champs);
  protected readonly tags = new FormArray<FormControl<string>>([]);
  protected readonly cree = signal<string | null>(null);

  protected ajouterTag(): void {
    this.tags.push(new FormControl('', { nonNullable: true, validators: Validators.required }));
  }

  protected retirerTag(index: number): void {
    this.tags.removeAt(index);
  }

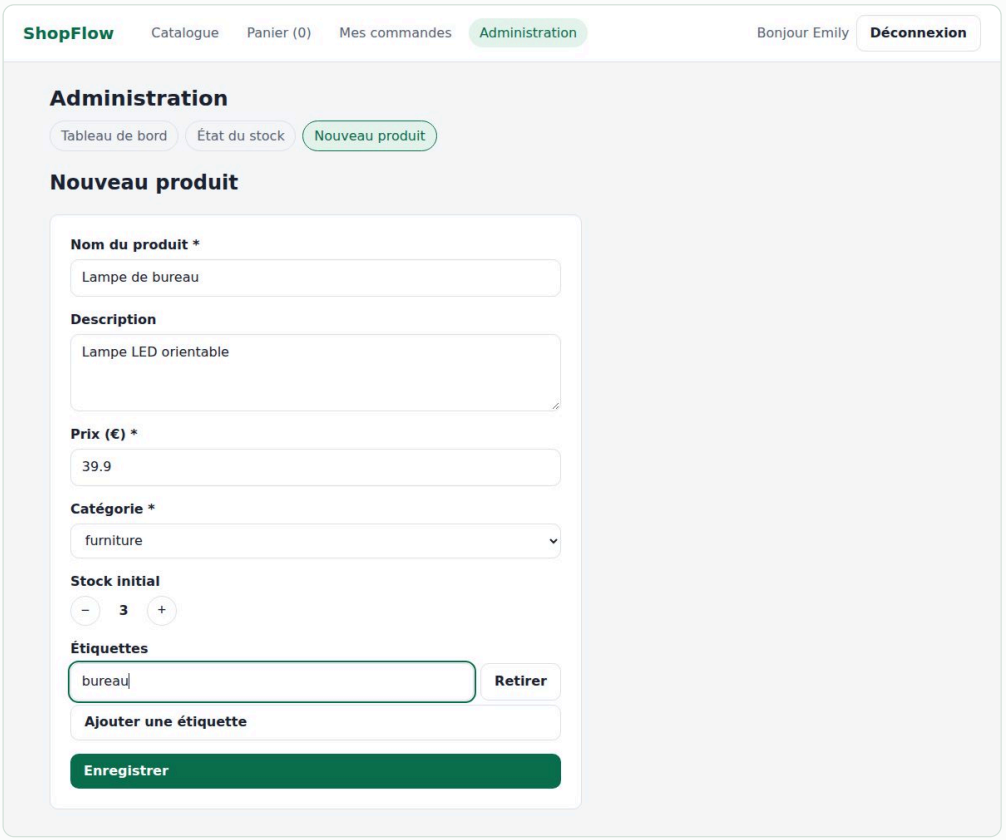
  protected enregistrer(): void {
    if (this.formulaire.invalid || this.tags.invalid) {
      this.formulaire.markAllAsTouched();
      this.tags.markAllAsTouched();
      return;
    }
    const produit = { ...this.formulaire.getRawValue(), tags: this.tags.getRawValue() };
    this.productService.add(produit).subscribe((reponse) => {
      this.cree.set(`Produit n° ${reponse.id} créé : ${reponse.title}`);
      this.formulaire.reset();
      this.tags.clear();
    });
  }
}
```

src/app/admin/product-form.html

```
<h2>Nouveau produit</h2>
```

```
<h2>Nouveau produit</h2>
@if (cree(); as message) {
  <p class="succes">{{ message }}</p>
}
<form class="formulaire" [formGroup]="formulaire" (ngSubmit)="enregistrer()">
  @for (champ of champs; track champ.cle) {
    @let controle = formulaire.controls[champ.cle];
    <div class="groupe">
      <label [for]="champ.cle">{{ champ.libelle }}{{ champ.requis ? ' *' : '' }}</label>
      @switch (champ.type) {
        @case ('zone') {
          <textarea class="champ" rows="3" [id]="champ.cle" [formControlName]="champ.cle"></textarea>
        }
        @case ('nombre') {
          <input class="champ" type="number" step="0.01" [id]="champ.cle" [formControlName]="champ.cle" />
        }
        @case ('liste') {
          <select class="champ" [id]="champ.cle" [formControlName]="champ.cle">
            <option value="">Choisir…</option>
            @for (o of champ.options; track o) {
              <option [value]="o">{{ o }}</option>
            }
          </select>
        }
        @case ('quantite') {
          <sf-quantite [id]="champ.cle" [formControlName]="champ.cle" />
        }
        @default {
          <input class="champ" [id]="champ.cle" [formControlName]="champ.cle" />
        }
      }
      @if (controle.touched && controle.hasError('required')) {
        <p class="erreur">Champ obligatoire.</p>
      } @else if (controle.touched && controle.hasError('min')) {
        <p class="erreur">Au moins {{ controle.getError('min').min }}.</p>
      }
    </div>
  }
  <div class="groupe">
    <span class="libelle">Étiquettes</span>
    @for (tag of tags.controls; track tag; let i = $index) {
      <div class="ligne-tag">
        <input class="champ" [formControl]="tag" [attr.aria-label]="'Étiquette ' + (i + 1)" />
        <button class="bouton bouton-secondaire" type="button" (click)="retirerTag(i)">Retirer</button>
      </div>
    }
    <button class="bouton bouton-secondaire" type="button" (click)="ajouterTag()">Ajouter une
étiquette</button>
  </div>
  <button class="bouton" type="submit">Enregistrer</button>
</form>
```

4. Le résultat



Le formulaire produit généré, avec le sélecteur de quantité et une étiquette.

10 h 45 – 11 h 00 : pause

11 h 15 – 11 h 45

Tester un pipe, un service, un guard, un composant

Module 7, « Karma et Jasmine, cas de test ».

1. La notion

Objectif : vérifier automatiquement que les parties sensibles de ShopFlow font ce qu'on attend.

Pourquoi : ShopFlow n'a aucun test : chaque modification peut casser une fonction sans que personne ne le voie.

Dans la réalité : un calcul de prix faux mis en production coûte de l'argent à chaque commande. Les tests tournent à chaque modification, dans l'intégration continue, avant la mise en ligne.

2. La démonstration

Terminal : shopflow (en continu pendant le développement, ou une seule fois) — npx nx test

```
npx nx test
npx nx test --watch=false
```

Résultat de départ : « Executed 0 of 0 », aucun test.

3. La correction

libs/shared/ui/src/lib/prix-remise/prix-remise-pipe.spec.ts — import { Product } from

```
import { Product } from '@shopflow/models';
import { PrixRemisePipe } from './prix-remise-pipe';

describe('PrixRemisePipe', () => {
  const pipe = new PrixRemisePipe();

  it('applique la remise en pourcentage', () => {
    const produit = { price: 100, discountPercentage: 12.5 } as Product;
    expect(pipe.transform(produit)).toBe(87.5);
  });

  it('arrondit au centime', () => {
    const produit = { price: 9.99, discountPercentage: 7.17 } as Product;
    expect(pipe.transform(produit)).toBe(9.27);
  });

  it('renvoie le prix tel quel sans remise', () => {
    expect(pipe.transform({ price: 20, discountPercentage: 0 } as Product)).toBe(20);
  });
});
```

libs/shared/data-access/src/lib/services/cart-service.spec.ts

```
import { TestBed } from
```

```ts
import { TestBed } from '@angular/core/testing';
import { Product } from '@shopflow/models';
import { CartService } from './cart-service';

const produit = (id: number, price: number): Product => ({ id, title: `Produit ${id}`, price,
discountPercentage: 0, thumbnail: '' }) as Product;

describe('CartService', () => {
  let panier: CartService;

  beforeEach(() => {
    localStorage.clear();
    TestBed.configureTestingModule({});
    panier = TestBed.inject(CartService);
  });

  it('ajoute un produit, puis augmente sa quantité', () => {
    panier.add(produit(1, 10));
    panier.add(produit(1, 10));
    expect(panier.lignes().length).toBe(1);
    expect(panier.nbArticles()).toBe(2);
    expect(panier.total()).toBe(20);
  });

  it('retire la ligne quand la quantité passe à 0', () => {
    panier.add(produit(1, 10));
    panier.setQuantity(1, 0);
    expect(panier.lignes()).toEqual([]);
  });

  it('sauvegarde le panier dans le localStorage', () => {
    panier.add(produit(2, 5));
    TestBed.tick();
    expect(JSON.parse(localStorage.getItem('shopflow-panier') ?? '[]').length).toBe(1);
  });
});
```

src/app/guards/auth-guard.spec.ts

```
import { TestBed } from
```

```ts
import { TestBed } from '@angular/core/testing';
import { ActivatedRouteSnapshot, provideRouter, Router, RouterStateSnapshot, UrlTree } from '@angular/router';
import { AuthService } from '@shopflow/data-access';
import { authGuard } from './auth-guard';

describe('authGuard', () => {
  const lancer = (token: string | null) => {
    TestBed.configureTestingModule({
      providers: [provideRouter([]), { provide: AuthService, useValue: { token } }],
    });
    return TestBed.runInInjectionContext(() =>
      authGuard({} as ActivatedRouteSnapshot, { url: '/mes-commandes' } as RouterStateSnapshot),
```

```
    );
  };

  it('laisse passer un utilisateur connecté', () => {
    expect(lancer('jeton')).toBeTrue();
  });

  it('redirige vers la connexion en mémorisant la page demandée', () => {
    const resultat = lancer(null) as UrlTree;
    expect(TestBed.inject(Router).serializeUrl(resultat)).toBe('/connexion?retour=%2Fmes-commandes');
  });
});
```

libs/shared/ui/src/lib/product-card/product-card.spec.ts

```
import { ComponentFixture, TestBed } from
```

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { provideRouter } from '@angular/router';
import { Product } from '@shopflow/models';
import { ProductCard } from './product-card';

describe('ProductCard', () => {
  let fixture: ComponentFixture<ProductCard>;
  const produit = { id: 7, title: 'Lampe', category: 'furniture', price: 50, discountPercentage: 10, rating:
4.5, thumbnail: 'https://exemple.test/lampe.webp' } as Product;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [ProductCard],
      providers: [provideRouter([])],
    }).compileComponents();
    fixture = TestBed.createComponent(ProductCard);
    fixture.componentRef.setInput('product', produit);
    await fixture.whenStable();
  });

  it('affiche le titre et le prix remisé', () => {
    const texte: string = fixture.nativeElement.textContent;
    expect(texte).toContain('Lampe');
    expect(texte).toContain('€45.00');
  });

  it('émet ajout avec le produit au clic', () => {
    let emis: Product | undefined;
    fixture.componentInstance.ajout.subscribe((p) => (emis = p));
    fixture.nativeElement.querySelector('button').click();
    expect(emis).toBe(produit);
  });
});
```

4. Le résultat

Tous les tests passent ; chacun décrit en une phrase ce qu'il vérifie, ce qui documente aussi le code.

11 h 45 – 12 h 15

Corriger un défaut en écrivant le test d'abord (TDD)

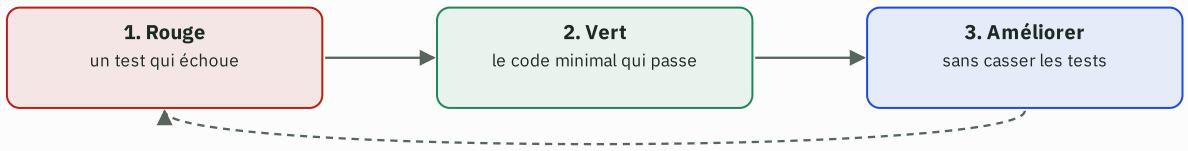
Module 7, « BDD, TDD ».

1. La notion

Objectif : corriger le prix faux de la fiche produit en commençant par un test qui le prouve.

Pourquoi : un défaut corrigé sans test peut revenir à la prochaine modification.

Dans la réalité : un client signale une erreur : on écrit d'abord un test qui la reproduit (il échoue), on corrige (il passe), et ce test empêche l'erreur de revenir.



Le cycle TDD.

2. La démonstration

src/app/pages/product-detail/product-detail.spec.ts

```
import { TestBed } from '@angular/core/testing';
import { provideRouter } from '@angular/router';
import { Product } from '@shopflow/models';
import { ProductDetail } from './product-detail';

describe('ProductDetail', () => {
  it('affiche le même prix que la carte du catalogue (prix remisé)', async () => {
    await TestBed.configureTestingModule({ imports: [ProductDetail], providers: [provideRouter([])]
}).compileComponents();
    const fixture = TestBed.createComponent(ProductDetail);
    const produit = { id: 7, title: 'Lampe', category: 'furniture', price: 50, discountPercentage: 10, rating:
4.5, stock: 3, description: '', thumbnail: 'https://exemple.test/lampe.webp', reviews: [] } as unknown as
Product;
    fixture.componentRef.setInput('produit', produit);
    await fixture.whenStable();
    const prix: string = fixture.nativeElement.querySelector('.detail-prix').textContent;
    expect(prix).toContain('€45.00');
  });
});
```

Terminal : shopflow (étape rouge : le test est écrit, la correction pas encore)

```
$ npx nx test --watch=false
ProductDetail affiche le même prix que la carte du catalogue (prix remisé) FAILED
        Expected '€50.00' to contain '€45.00'.
Executed 12 of 12 (1 FAILED)
TOTAL: 1 FAILED, 11 SUCCESS
```

Rouge : la fiche affiche 50 € au lieu de 45 € ; le défaut constaté lors de la visite est confirmé par un test.

3. La correction

src/app/pages/product-detail/product-detail.html (extrait) (extrait) <p class="detail-prix">

```
<p class="detail-prix">
  {{ p | prixRemise | currency: 'EUR' }}
  @if (p.discountPercentage > 0) {
    <s class="prix-barre">{{ p.price | currency: 'EUR' }}</s>
  }
</p>
```

4. Le résultat

Terminal : shopflow (étape verte, avec couverture)

```
$ npx nx test --watch=false --coverage
Executed 12 of 12 SUCCESS (0.121 secs / 0.091 secs)
TOTAL: 12 SUCCESS
------------------------------ Coverage summary ------------------------------
Statements   : 74.68% ( 59/79 )
Branches     : 87.5% ( 14/16 )
Functions    : 57.14% ( 4/7 )
Lines        : 74.35% ( 29/39 )
------------------------------------------------------------------------------
```

Vert : 12 tests réussis.

12 h 15 – 12 h 30

Mesurer la couverture des tests

Module 7, « Couverture de code ».

1. La notion

Objectif : savoir quelles lignes de code sont exécutées par les tests, et lesquelles ne le sont jamais.

Pourquoi : des tests qui passent ne disent pas ce qui n'est pas testé.

Dans la réalité : beaucoup d'entreprises fixent un seuil (par exemple 70 %) en dessous duquel une modification est refusée.

2. La démonstration

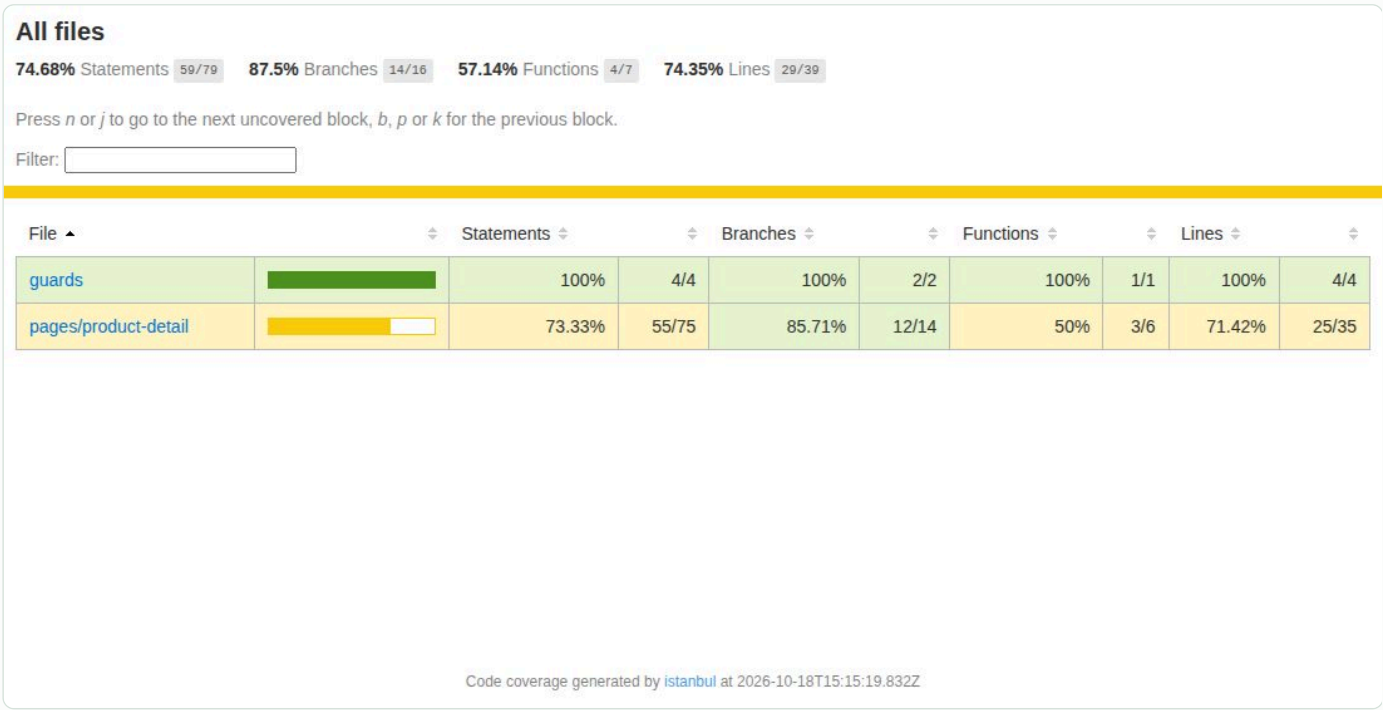
Terminal : shopflow — `npx nx test --watch=false --coverage`

```
npx nx test --watch=false --coverage
```

3. La mise en œuvre

Rien à corriger : on lit le rapport et on repère les fichiers les moins couverts.

4. Le résultat



Le rapport HTML, dans coverage/shopflow/index.html : on clique sur un fichier pour voir les lignes non exécutées.

12 h 30 – 13 h 30 : déjeuner

13 h 30 – 13 h 45

Tester un parcours complet avec Playwright

Module 7, « Tests d'intégration » (Protractor, abandonné, remplacé par Playwright).

1. La notion

Objectif : vérifier un parcours réel dans un vrai navigateur : se connecter, ouvrir Mes commandes.

Pourquoi : les tests unitaires vérifient des morceaux ; ils ne voient pas une redirection cassée ou un bouton qui ne mène nulle part.

Dans la réalité : avant chaque mise en ligne, quelques parcours essentiels (connexion, commande) sont rejoués automatiquement.

2. La démonstration

Protractor, cité par le programme, n'existe plus depuis 2023 ; on utilise Playwright.

Terminal : shopflow — npm install --save-dev @playwright/test

```
npm install --save-dev @playwright/test
npx playwright install chromium
```

3. La correction

playwright.config.ts — import { defineConfig, devices } from

```ts
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './e2e',
  use: { baseURL: 'http://localhost:4200', trace: 'on-first-retry' },
  webServer: {
    command: 'npx nx serve',
    url: 'http://localhost:4200',
    reuseExistingServer: true,
    timeout: 120_000,
  },
  projects: [{ name: 'chromium', use: { ...devices['Desktop Chrome'] } }],
});
```

e2e/parcours.spec.ts — import { expect, test } from

```ts
import { expect, test } from '@playwright/test';

test('un client se connecte et retrouve ses commandes', async ({ page }) => {
  await page.goto('/mes-commandes');
  await expect(page).toHaveURL(/\/connexion\?retour=%2Fmes-commandes/);
  await page.getByLabel('Identifiant').fill('emilys');
  await page.getByLabel('Mot de passe').fill('emilyspass');
```

```
  await page.getByRole('button', { name: 'Se connecter' }).click();
  await expect(page).toHaveURL('/mes-commandes');
  await expect(page.getByRole('heading', { name: 'Commande n° 1' })).toBeVisible();
});

test('un modérateur est renvoyé au catalogue', async ({ page }) => {
  await page.goto('/admin');
  await page.getByLabel('Identifiant').fill('oliviaw');
  await page.getByLabel('Mot de passe').fill('oliviawpass');
  await page.getByRole('button', { name: 'Se connecter' }).click();
  await expect(page).toHaveURL('/catalogue');
});

test('une commande incomplète est refusée', async ({ page }) => {
  await page.goto('/catalogue');
  await page.getByRole('button', { name: 'Ajouter au panier' }).first().click();
  await page.goto('/commande');
  await page.getByRole('button', { name: 'Valider la commande' }).click();
  await expect(page.getByText('Le nom est obligatoire.')).toBeVisible();
  await expect(page).toHaveURL('/commande');
});
```

4. Le résultat

Terminal : shopflow

```
$ npx playwright test --reporter=list
Running 3 tests using 1 worker
  ✓  1 [chromium] › e2e/parcours.spec.ts:3:5 › un client se connecte et retrouve ses commandes (2.3s)
  ✓  2 [chromium] › e2e/parcours.spec.ts:13:5 › un modérateur est renvoyé au catalogue (1.8s)
  ✓  3 [chromium] › e2e/parcours.spec.ts:21:5 › une commande incomplète est refusée (1.7s)
  3 passed (17.5s)
```

Trois parcours réels, sur l'application et l'API : guards, connexion, formulaire.

13 h 45 – 14 h 30

Envoyer des pages déjà rendues par le serveur

Module 8, « Rendu côté serveur ».

1. La notion

Objectif : que le serveur envoie une page HTML complète, lisible sans JavaScript.

Pourquoi : aujourd'hui, le serveur envoie une balise `<app-root>` vide : un moteur de recherche ou un aperçu de lien ne voit aucun produit.

Dans la réalité : une boutique dont les fiches n'apparaissent pas dans Google perd une grande partie de ses visiteurs. Le rendu serveur affiche aussi la page plus vite sur un téléphone lent.



Trois modes, choisis route par route selon les données affichées.

2. La démonstration

Terminal 2 – n'importe quel dossier (la version de production tourne sur le port 8080)

```
curl -s http://127.0.0.1:8080/catalogue |
```

```
curl -s http://127.0.0.1:8080/catalogue | grep -o "<app-root>.*</app-root>"
```

Résultat : `<app-root></app-root>`, aucun produit.

3. La correction

Terminal : shopflow (dans un projet Angular CLI sans Nx : ng add @angular/ssr)

```
npx nx g @nx/angular:setup-ssr
```

```
npx nx g @nx/angular:setup-ssr --project=shopflow
```

src/app/app.routes.server.ts — import { RenderMode, ServerRoute } from

```ts
import { RenderMode, ServerRoute } from '@angular/ssr';

export const serverRoutes: ServerRoute[] = [
  { path: 'catalogue', renderMode: RenderMode.Prerender },
  { path: 'catalogue/:id', renderMode: RenderMode.Server },
  { path: 'panier', renderMode: RenderMode.Client },
  { path: 'commande', renderMode: RenderMode.Client },
  { path: 'commande/confirmation', renderMode: RenderMode.Client },
  { path: 'connexion', renderMode: RenderMode.Client },
  { path: 'mes-commandes', renderMode: RenderMode.Client },
  { path: 'admin/**', renderMode: RenderMode.Client },
  { path: '**', renderMode: RenderMode.Server },
];
```

Le code qui ne doit tourner que dans le navigateur (le `localStorage` n'existe pas sur le serveur) :

libs/shared/data-access/src/lib/services/cart-service.ts (extrait) (extrait) — import { computed, effect, inject,

```ts
import { computed, effect, inject, PLATFORM_ID, Service, signal } from '@angular/core';
import { isPlatformBrowser } from '@angular/common';

@Service()
export class CartService {
  private readonly navigateur = isPlatformBrowser(inject(PLATFORM_ID));
  private readonly etat = signal<CartLine[]>(this.navigateur ? lire() : []);

  constructor() {
    effect(() => {
      const lignes = this.etat();
      if (this.navigateur) {
        localStorage.setItem(CLE_STOCKAGE, JSON.stringify(lignes));
      }
    });
  }
```

4. Le résultat

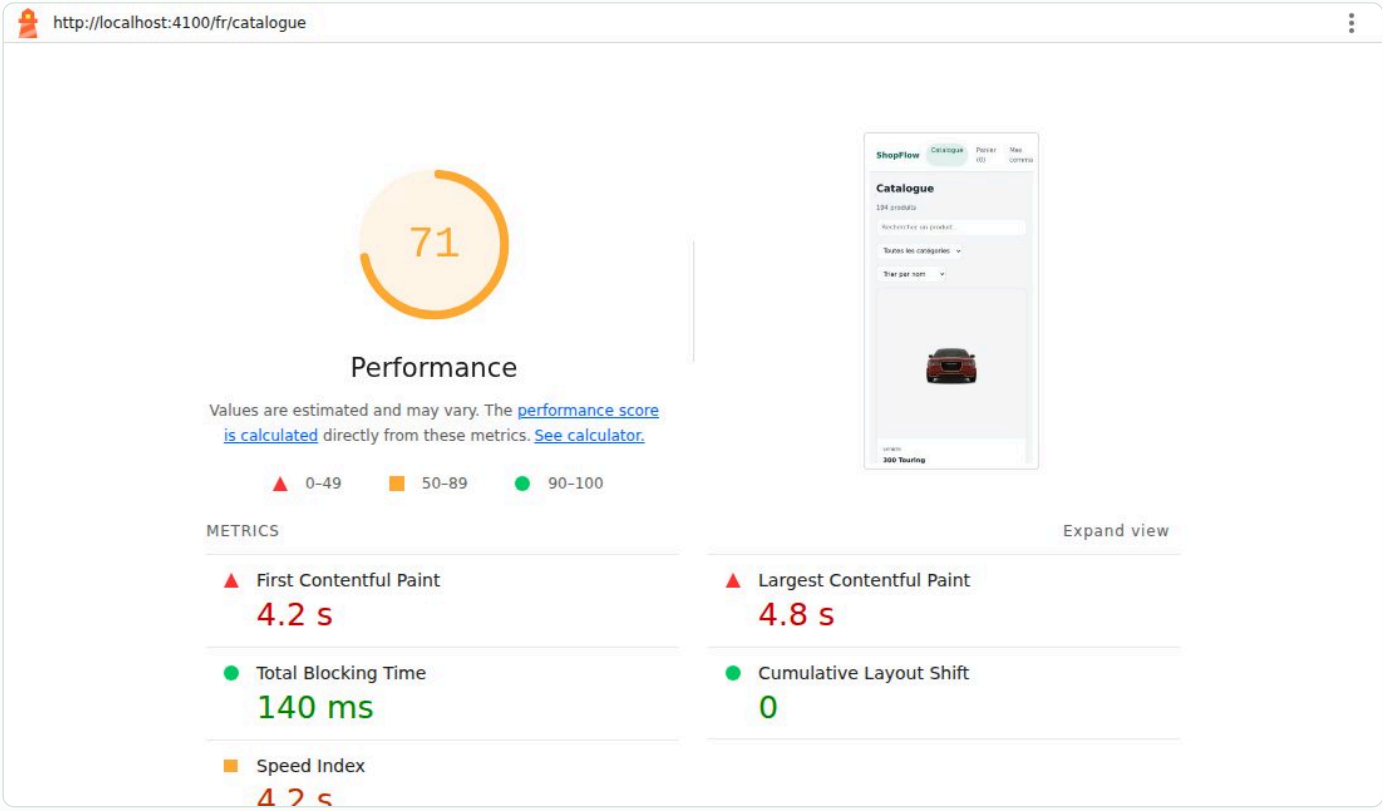
Terminal : shopflow — npx nx build

```
npx nx build
PORT=4000 node dist/shopflow/server/server.mjs
```

Terminal : shopflow

```
$ PORT=4000 node dist/shopflow/server/server.mjs
Node Express server listening on http://localhost:4000
$ curl -s http://localhost:4000/fr/catalogue | grep -o "<sf-product-card" | wc -l
194
$ curl -s http://localhost:4000/fr/catalogue/12 | grep -o "<h1[^>]*>[^<]*</h1>"
<h1>Annibale Colombo Sofa</h1>
$ curl -s http://localhost:4000/fr/panier | grep -c "Mon panier"
0
$ curl -s -o /dev/null -w "%{http_code} %{redirect_url}" http://localhost:4000/
302 http://localhost:4000/fr
```

Le catalogue pré-rendu contient ses 194 cartes ; la fiche est rendue à la demande ; le panier reste rendu dans le navigateur ; la racine redirige vers la langue du visiteur.



Lighthouse sur le catalogue pré-rendu, champs limités.

14 h 30 – 14 h 45

Séquence 17 : bilan

Au programme de cette séquence (15 min)

- 5 min : les mesures du matin du jour 1 et celles d'aujourd'hui
- 5 min : l'état du cahier des charges
- 5 min : questions, présentation du TP final

Objectif : refermer la démarche ouverte à la séquence 2 (mesurer, comprendre, corriger, re-mesurer) avec des chiffres, et dire honnêtement ce qui reste à faire.

1. Avant, après

Mesure	Jour 1, matin	Jour 3, fin	Séquence
Lighthouse mobile, médiane de 3 mesures	42	71 (rendu serveur)	5, 16
Largest Contentful Paint	8,3 s	4,8 s	5, 16
Total Blocking Time	5 410 ms	140 à 340 ms	5, 16
Requêtes au chargement du catalogue	199	14 (mobile)	5
Images téléchargées à l'ouverture (ordinateur)	194	32	5
Calculs de la liste pendant la saisie de `phone`	5	1	9, 11
Calcul du prix pendant la saisie	88 appels	0	7
Requêtes du catalogue complet par visite	3	1 (0 avec le rendu serveur)	9, 16
Requêtes après avoir quitté l'administration	6 en 30 s, cumulées	0	9
Tests automatiques	0	12 unitaires, 3 de bout en bout	15
HTML reçu par un moteur de recherche	`<app-root>` vide	194 produits	16

2. Le cahier des charges

N°	Besoin	État	Preuve
1	Premier affichage limité au catalogue	Validé	Pages en « Lazy chunk files » (séquence 5)
2	Images à l'approche de l'écran	Validé	32 images au lieu de 194 ; Lighthouse ne signale plus les images
3	Recalcul en fin de saisie	Validé	1 calcul pour `phone` (séquences 9 et 11)
4	Bibliothèques réutilisables	Validé	Graphe Nx : `ui` ne dépend pas de `data-access`
5	Code documenté	Commencé	Compodoc en place, couverture 8 % : à poursuivre
6	Catalogue demandé une fois	Validé	1 requête ; test `ProductService`
7	Plus de requêtes après l'administration	Validé	0 requête en 20 s
8	Pages privées réservées	Validé	Guards ; 2 parcours Playwright
9	Panier simple à lire	Validé	`CartService` en signaux ; 3 tests
10	Français et anglais	En grande partie	Catalogue, en-tête, commandes, connexion ; commande et administration à marquer
11	Commande incomplète refusée	Validé	Validation réactive ; parcours Playwright
12	Code testé	Validé	12 tests unitaires, 3 de bout en bout, couverture mesurée
13	Même prix carte et fiche	Validé	Test écrit avant la correction (TDD)

N°	Besoin	État	Preuve
14	Catalogue lisible par un moteur de recherche	Validé	HTML pré-rendu de 194 produits
15	Rapide sur mobile	Amélioré	42 → 71 ; pistes : pagination du catalogue, rendu serveur des premiers produits

> **Un avertissement encore présent**
>
> En grand écran, la console affiche `NG02955` : l'image la plus visible n'est pas marquée prioritaire. Sur un écran large, huit cartes sont visibles alors que seules les quatre premières sont prioritaires. Deux réponses possibles, à discuter : passer à `$index < 8` (plus d'images en concurrence sur téléphone), ou calculer le nombre de colonnes. C'est un avertissement de performance, pas une erreur : la page fonctionne.

3. Le code, journée par journée

Le code de ShopFlow et de Cuisto est fourni dans `ressources/` : `depart/` pour l'état initial, puis `jour-1/`, `jour-2/` et `jour-3/` pour l'état attendu en fin de chaque journée. Un apprenant en retard repart du dossier de la veille.

4. Questions de clôture

Le TP final applique la même démarche à une autre application, seuls ou en binômes : diagnostic, cahier des charges, corrections, mesures.

14 h 45 – 15 h 15

Rattrapage

Finir les étapes indispensables des TP précédents, questions.

> **Temps de rattrapage (30 min)**
>
> Avant le dernier TP : finir les étapes **indispensables** des TP des jours 1 et 2 sur Cuisto, poser les questions restées en suspens, vérifier que chaque poste construit Cuisto sans erreur (`npx nx build cuisto`).
>
> Celui qui n'a pas pu finir le jour 2 repart de `cuisto-debut-jour-3.zip` (dossier `ressources/points-de-reprise/`), avec les mêmes commandes que la reprise du jour 2.

15 h 15 – 15 h 30 : pause

15 h 30 – 16 h 45

TP Cuisto, jour 3

Objectif du TP

À la fin du TP : Cuisto existe en anglais pour cinq textes, le formulaire « Proposer une recette » refuse un envoi incomplet, et un test prouve que la durée affichée sur la fiche est juste.

Pourquoi : ce sont les trois attentes les plus courantes en entreprise sur une application : plusieurs langues, des saisies contrôlées, et des corrections prouvées par des tests.

À rendre : le dépôt avec l'étiquette `jour-3` à 16 h 45 ; il est évalué tel quel.

Copier les commandes depuis la version HTML de ce document (bouton **Copier**) ou depuis le fichier de commandes, jamais depuis le PDF. Sous Windows, utiliser **Git Bash**.

Point de départ : votre Cuisto de la veille, ou `cuisto-debut-jour-3.zip` fourni par le formateur (dossier `ressources/points-de-reprise/`).

1. Traduire cinq textes (25 min)

À obtenir : une version anglaise de Cuisto où les liens du menu et le titre de la liste sont traduits.

Terminal — dans cuisto/ (racine du projet, là où se trouve package.json) `npx nx add @angular/localize`

```
npx nx add @angular/localize
```

Configurer les langues : dans `cuisto/project.json` , ajouter le bloc `i18n` au premier niveau, et `"localize": true` dans la configuration `production` de la cible `build` .

cuisto/project.json (bloc à ajouter) (extrait) "i18n": {

```
"i18n": {
  "sourceLocale": "fr",
  "locales": {
    "en": { "translation": "src/locale/messages.en.xlf" }
  }
},
```

Marquer les textes : ajouter un attribut `i18n` avec un identifiant aux trois liens du menu (`src/app/layout/header/header.html`), au titre et au champ de recherche de la liste (`src/app/pages/recipes/recipes.html`). Par exemple : `<h1 i18n="@@recettes.titre">Recettes</h1>`.

Terminal — dans cuisto/ (racine du projet, là où se trouve package.json) npx nx run cuisto:extract-i18n

```
npx nx run cuisto:extract-i18n --output-path=src/locale
cp src/locale/messages.xlf src/locale/messages.en.xlf
```

Dans `messages.en.xlf`, ajouter une balise `<target>` avec la traduction sous chaque `<source>`.
Puis construire et servir :

Terminal — dans cuisto/ (racine du projet, là où se trouve package.json) npx nx build cuisto

```
npx nx build cuisto
npx serve -s dist/cuisto/browser -l 8081
```

> **Vérifier** : `http://127.0.0.1:8081/en/recettes` affiche les cinq textes en anglais, `http://127.0.0.1:8081/fr/recettes` en français.

2. Contrôler le formulaire (25 min)

À obtenir : le bouton « Envoyer » de « Proposer une recette » reste désactivé tant que le nom fait moins de trois caractères, avec un message d'erreur.

Fichiers : `src/app/admin/recipe-form.ts` et `.html`.

À faire : remplacer `FormsModule` et `ngModel` par un formulaire réactif (`ReactiveFormsModule`) : un `FormGroup` créé avec `NonNullableFormBuilder`, dont le champ `name` porte `Validators.required` et `Validators.minLength(3)`. Dans le template : `[formGroup]`, `formControlName`, un message affiché quand le champ est invalide et a été touché, et `[disabled]="form.invalid"` sur le bouton.

> **Vérifier** : nom vide ou de deux lettres : bouton grisé et message ; trois lettres : bouton actif.

3. Prouver la durée de la fiche par un test (20 min)

À obtenir : un test qui échoue d'abord, puis passe après la correction : pour une recette de 30 min de préparation et 45 min de cuisson, la fiche affiche « 1 h 15 min ».

Constat : la fiche affiche « Temps total : 30 min », la carte « 1 h 15 min ».

À faire, dans cet ordre : écrire le test dans `src/app/pages/recipe-detail/recipe-detail.spec.ts` ; le lancer : il échoue ; corriger le template de la fiche en réutilisant le pipe de durée de la carte ; relancer : il passe. Enregistrer un commit après chaque étape, pour que l'ordre se voie dans l'historique.

Terminal — dans cuisto/ (racine du projet, là où se trouve package.json) `npx nx test cuisto --watch=false`

```
npx nx test cuisto --watch=false
```

Indice : `TestBed.createComponent(RecipeDetail)`, puis donner une recette au composant (`fixture.componentRef.setInput(…)`), `await fixture.whenStable()`, et lire le texte de l'élément `.detail-prix`.

4. Rendre (5 min)

À obtenir : le dépôt étiqueté avant 16 h 45.

Terminal — dans cuisto/ (racine du projet, là où se trouve package.json) `git add -A`

```
git add -A
git commit -m "Jour 3"
git tag jour-3
git push --follow-tags
```

Pour aller plus loin (facultatif)

À faire seulement si les étapes précédentes sont terminées. Chaque point donne des points bonus.

- Pluriels (« 1 recette », « 2 recettes ») et difficulté traduite (`plural`, `select`).
- Ingrédients du formulaire en liste de groupes (`FormArray`) ; validateur asynchrone sur le nom.
- Deux parcours de bout en bout avec Playwright.
- Rendu côté serveur et pré-rendu des fiches.

Si le temps le permet

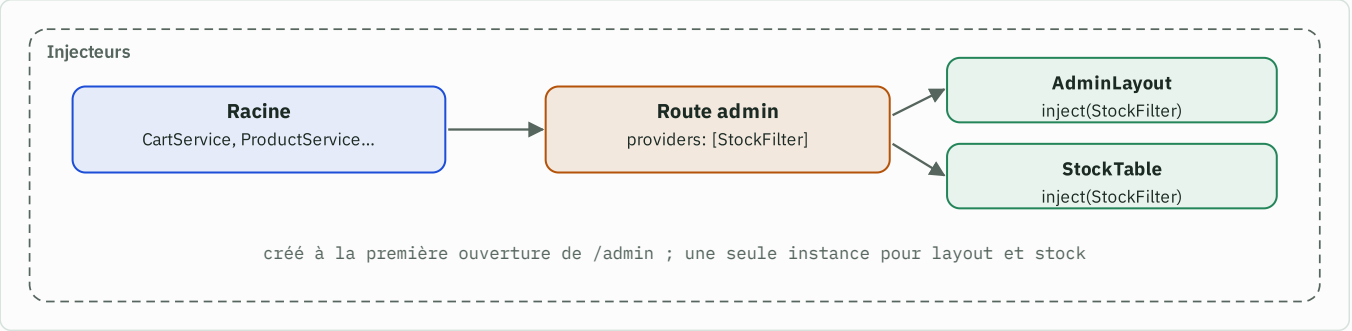
Pour aller plus loin

Un service fourni par une route

3. Un service fourni par une route (10 min)

Le besoin : l'administration veut filtrer le tableau du stock, et garder ce filtre quand on passe du tableau de bord au stock. Ce filtre ne concerne que l'administration : inutile de le créer, ni même de le télécharger, pour un visiteur du catalogue.

Avec les NgModules, on aurait mis ce service dans le tableau `providers` du module de l'administration. Sans module, c'est la **route** qui le fournit : toutes les pages enfants de `admin` reçoivent la même instance, distincte du reste de l'application.



La route admin a son propre injecteur : StockFilter n'existe que là.

src/app/admin/stock-filter.ts `import { computed, Injectable, signal }`

```ts
import { computed, Injectable, signal } from '@angular/core';

/** Filtre du tableau du stock, propre à l'espace d'administration. */
@Injectable()
export class StockFilter {
  readonly terme = signal('');
  readonly seulementFaible = signal(false);
  readonly actif = computed(() => this.terme() !== '' || this.seulementFaible());

  constructor() {
    console.log('StockFilter : instance créée');
  }
}
```

src/app/admin/admin.routes.ts `import { Routes } from '@angular/router';`

```ts
import { Routes } from '@angular/router';
import { adminGuard } from '../guards/admin-guard';
import { AdminLayout } from './admin-layout';
import { AdminDashboard } from './admin-dashboard';
import { StockTable } from './stock-table';
import { StockFilter } from './stock-filter';

export const adminRoutes: Routes = [
  {
    path: '',
    component: AdminLayout,
    canActivateChild: [adminGuard],
    providers: [StockFilter],
    children: [
      { path: '', component: AdminDashboard },
      { path: 'stock', component: StockTable },
    ],
  },
];
```

src/app/admin/stock-table.ts `import { Component, computed, inject }`

```ts
import { Component, computed, inject } from '@angular/core';
import { CurrencyPipe } from '@angular/common';
import { toSignal } from '@angular/core/rxjs-interop';
```

```ts
import { ProductService } from '@shopflow/data-access';
import { BadgeStock } from '@shopflow/ui';
import { StockFilter } from './stock-filter';

@Component({
  selector: 'app-stock-table',
  imports: [BadgeStock, CurrencyPipe],
  templateUrl: './stock-table.html',
})
export class StockTable {
  protected readonly filtre = inject(StockFilter);
  private readonly page = toSignal(inject(ProductService).getAll());
  protected readonly lignes = computed(() => {
    const terme = this.filtre.terme().toLowerCase();
    return (this.page()?.products ?? []).filter(
      (p) => p.title.toLowerCase().includes(terme) && (!this.filtre.seulementFaible() || p.stock < 10),
    );
  });
}
```

src/app/admin/stock-table.html `<p class="barre-outils">`

```html
<p class="barre-outils">
  <input class="champ" type="search" placeholder="Filtrer les produits…" [value]="filtre.terme()"
(input)="filtre.terme.set($any($event.target).value)" />
  <label><input type="checkbox" [checked]="filtre.seulementFaible()"
(change)="filtre.seulementFaible.set($any($event.target).checked)" /> Stock faible seulement</label>
</p>
@if (lignes().length > 0) {
  <div class="table">
    <table class="tableau">
      <thead>
        <tr><th>Produit</th><th>Catégorie</th><th>Prix</th><th>Stock</th><th>Valeur</th><th>État</th></tr>
      </thead>
      <tbody>
        @for (p of lignes(); track p.id) {
          <tr>
            <td>{{ p.title }}</td>
            <td>{{ p.category }}</td>
            <td>{{ p.price | currency: 'EUR' }}</td>
            <td>{{ p.stock }}</td>
            <td>{{ p.price * p.stock | currency: 'EUR' }}</td>
            <td><sf-badge-stock [stock]="p.stock" /></td>
          </tr>
        }
      </tbody>
    </table>
  </div>
} @else {
  <p class="info">Aucun produit.</p>
}
```

src/app/admin/admin-layout.ts

```
import { Component, inject } from
```

```ts
import { Component, inject } from '@angular/core';
import { RouterLink, RouterLinkActive, RouterOutlet } from '@angular/router';
import { StockFilter } from './stock-filter';

@Component({
  selector: 'app-admin-layout',
  imports: [RouterLink, RouterLinkActive, RouterOutlet],
  templateUrl: './admin-layout.html',
})
export class AdminLayout {
  protected readonly filtre = inject(StockFilter);
}
```

src/app/admin/admin-layout.html

```
<h1>Administration</h1>
```

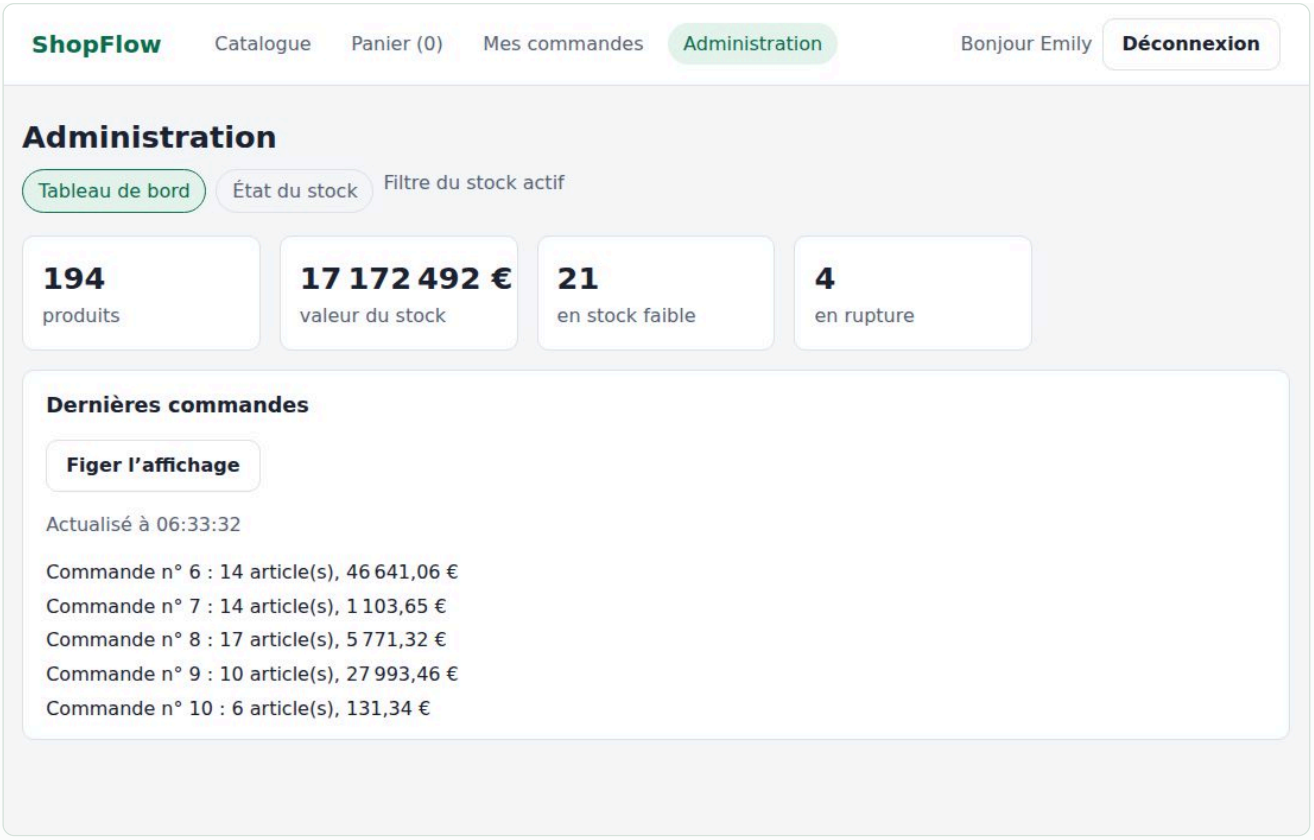
```html
<h1>Administration</h1>
<nav class="sous-nav">
  <a routerLink="/admin" routerLinkActive="actif" [routerLinkActiveOptions]="{ exact: true }">Tableau de bord</a>
  <a routerLink="/admin/stock" routerLinkActive="actif">État du stock</a>
  @if (filtre.actif()) {
    <span class="info">Filtre du stock actif</span>
  }
</nav>
<router-outlet />
```

Terminal : shopflow — se connecter (emilys / emilyspass), ouvrir http://localhost:4200/admin/stock, console ouverte (F12)

```
npm start
```

```
npm start
```

Vérification	Résultat obtenu
Message « StockFilter : instance créée » avant d'ouvrir l'administration	Aucun
À l'ouverture de /admin	Un message : l'instance est créée à ce moment-là
Filtre « table » dans le stock	2 lignes sur 194
Retour au tableau de bord	« Filtre du stock actif » dans le sous-menu : même instance
Catalogue, puis retour au stock	Le champ contient toujours « table » ; toujours une seule instance



Le tableau de bord indique que le filtre du stock est actif : layout et tableau partagent la même instance.

À savoir

- Une instance fournie par une route vit tant que l'application tourne : quitter `/admin` ne la détruit pas (le filtre est retrouvé au retour).
- Injecter `StockFilter` hors de l'administration (dans le catalogue, par exemple) provoque une erreur `NullInjectorError` : aucun injecteur ne le fournit.
- Le code du service est dans le fichier de l'administration, chargé à la demande : il n'alourdit pas le démarrage.

Migrer du code existant vers le standalone

4. Migrer du code existant

ShopFlow n'a rien à migrer. Dans une application existante, Angular fournit une migration officielle en trois étapes, à lancer dans l'ordre, en vérifiant que l'application compile entre chaque étape. On la montre sur un petit dossier `legacy/` (le module et le composant ci-dessus, avec un `*ngIf`), dans une copie du projet.

Terminal : projet d'exemple (code hérité)

```
$ npx nx g @angular/core:standalone-migration --mode=convert-to-standalone --path=src/app/legacy
NX  Generating @angular/core:standalone-migration
    🎉 Automated migration step has finished! 🎉
    IMPORTANT! Please verify manually that your application builds and behaves as expected.
    See https://angular.dev/reference/migrations/standalone for more information.
UPDATE src/app/legacy/promo-banner.component.ts
UPDATE src/app/legacy/prix.pipe.ts
UPDATE src/app/legacy/legacy.module.ts
$ npx nx g @angular/core:standalone-migration --mode=prune-ng-modules --path=src/app/legacy
DELETE src/app/legacy/legacy.module.ts
$ npx nx g @angular/core:control-flow --path=src/app/legacy
UPDATE src/app/legacy/promo-banner.component.ts
```

Étape 1 : les déclarations deviennent standalone. Étape 2 : le module devenu inutile est supprimé. Puis la conversion de *ngIf en @if.

Mode	Ce qu'il fait
`convert-to-standalone`	Retire `standalone: false`, ajoute à chaque composant les `imports` dont son template a besoin, déplace les déclarations du module dans ses `imports`
`prune-ng-modules`	Supprime les modules qui ne servent plus qu'à réexporter
`standalone-bootstrap`	Remplace `bootstrapModule(AppModule)` par `bootstrapApplication` et les modules du routeur, de HTTP… par les fonctions `provideXxx()`

src/app/legacy/promo-banner.component.ts (après les trois commandes)

```ts
import { Component, Input } from '@angular/core';

import { PrixPipe } from './prix.pipe';

@Component({
    selector: 'app-promo-banner',
    template: `@if (actif) {<p class="promo">Promotion : {{ montant | prix }}</p>}`,
    imports: [PrixPipe],
})
export class PromoBanner {
  @Input() actif = false;
  @Input() montant = 0;
}
```

Ce qu'il reste à faire à la main

La migration ne reformate pas le code (indentation de 4 espaces, ligne vide laissée) : on lance ensuite le formateur du projet. Elle ne touche pas aux tests ni aux `@Input` décorés. Et elle demande, en fin d'exécution, de vérifier que l'application se construit et fonctionne : c'est indispensable.

Exercice : traduire Mes commandes

5. Exercice (15 min)

Énoncé

Internationaliser la page « Mes commandes » : le titre, le message de chargement, le résumé (avec un pluriel pour le nombre de commandes), le titre de chaque commande, le nombre d'articles (avec un pluriel) et les deux messages d'erreur écrits dans `orders.ts`. Extraire, traduire, construire, vérifier en anglais.

▶ **Afficher le corrigé**

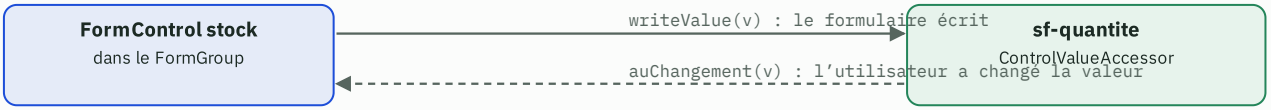
Un composant de formulaire sur mesure

5. TP : un composant métier de formulaire (20 min)

Énoncé

Le champ « Stock initial » doit utiliser un sélecteur « − valeur + » plutôt qu'un champ texte. Créer dans la bibliothèque `ui` un composant `sf-quantite` qui s'utilise **comme un champ de formulaire** : `formControlName`, `formControl` ou `ngModel` doivent fonctionner, ainsi que `disable()` et `reset()`. Indice : l'interface `ControlValueAccessor` et le jeton `NG_VALUE_ACCESSOR`.

▶ **Afficher le corrigé**



Les deux sens de communication entre un contrôle et un composant ControlValueAccessor.

Vérification	Résultat
Enregistrer sans rien remplir	« Champ obligatoire » (nom, catégorie), « Au moins 0.01 » (prix)
Trois clics sur « + »	Stock à 3, valeur du contrôle à 3
Formulaire complet, une étiquette	« Produit n° 195 créé : Lampe de bureau »

Exercice : choisir les modes de rendu

6. Exercice (10 min)

Énoncé

Pré-rendre à la construction les fiches des 10 premiers produits ; les autres fiches restent rendues à la demande. Vérifier le nombre de routes pré-rendues dans la sortie de `npx nx build`.

▶ **Afficher le corrigé**